

PRACTICAL 1.

Installation of java, android framework

Android Studio is the official IDE (Integrated Development Environment) for Android app development and it is based on JetBrains' IntelliJ IDEA software. Android Studio provides many excellent features that enhance productivity when building Android apps, such as:

- A blended environment where one can develop for all Android devices .
- Apply Changes to push code and resource changes to the running app without restarting the app.
- A flexible Gradle-based build system.
- A fast and feature-rich emulator.
- GitHub and Code template integration to assist you to develop common app features and import sample code.
- Extensive testing tools and frameworks.
- C++ and NDK support.
- Built-in support for Google Cloud Platform, making it easy to integrate Google Cloud Messaging and App Engine, and many more.
- Provides GUI tools that simplify the less interesting parts of app development.
- Easy integration with real time database 'firebase'.

System Requirements

- Microsoft Windows 7/8/10 (32-bit or 64-bit).
- 4 GB RAM minimum, 8 GB RAM recommended (plus 1 GB for the Android Emulator).
- 2 GB of available disk space minimum, 4 GB recommended (500 MB for IDE plus 1.5 GB for Android SDK and emulator system image).
- 1280 x 800 minimum screen resolution.

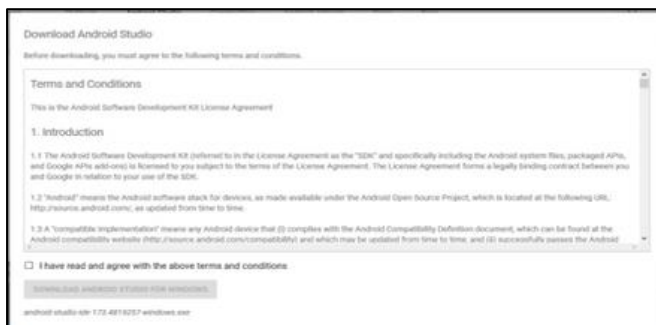
Installation Guide

Step 1: Head over to this link to get the Android Studio executable or zip file.

Step 2: Click on the Download Android Studio Button.



Click on the “I have read and agree with the above terms and conditions” checkbox followed by the download button.



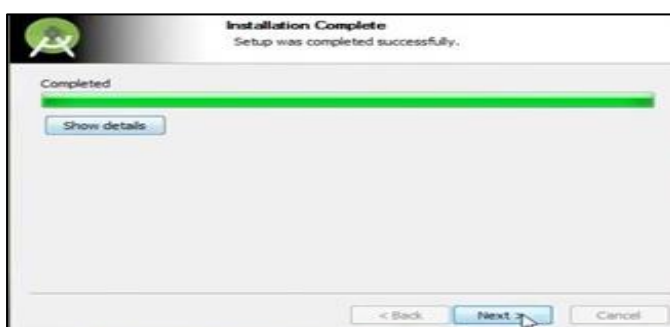
Click on the Save file button in the appeared prompt box and the file will start downloading.

Step 3: After the downloading has finished, open the file from downloads and run it. It will prompt the following dialog box.



Click on next. In the next prompt, it'll ask for a path for installation. Choose a path and hit next.

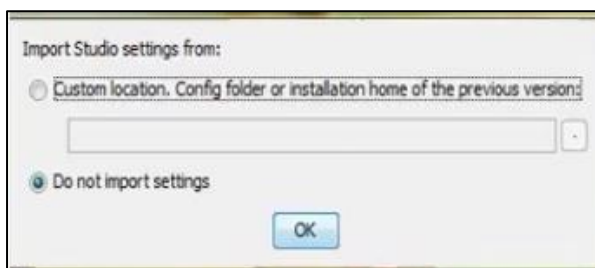
Step 4: It will start the installation, and once it is completed, it will be like the image shown below:



Click on next.



Step 5: Once “Finish” is clicked, it will ask whether the previous settings need to be imported [if the android studio had been installed earlier], or not. It is better to choose the ‘Don’t import Settings option’.



Click the OK button.

Step 6: This will start the Android Studio.

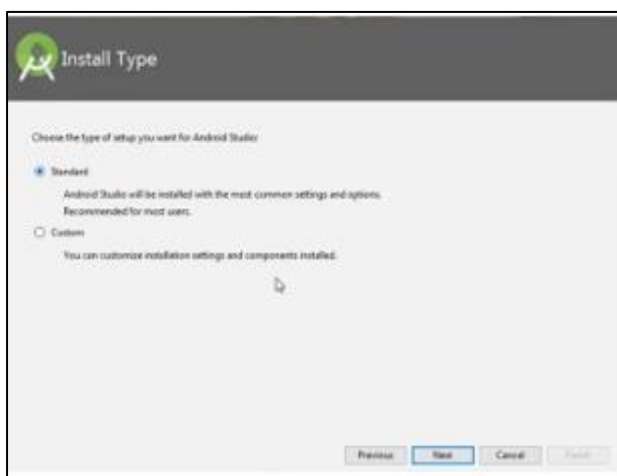


Meanwhile, it will be finding the available SDK components.

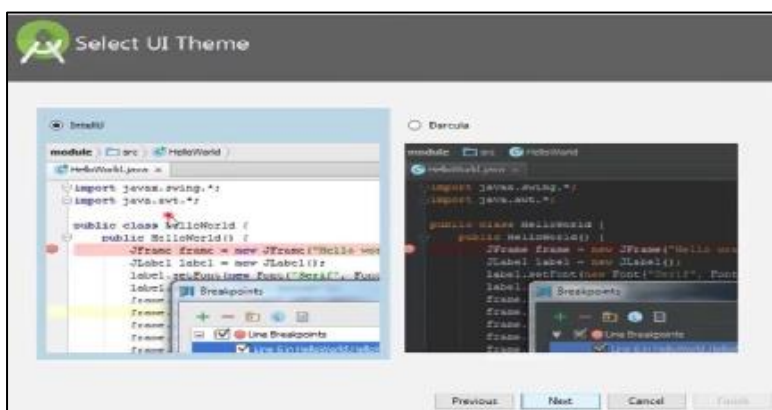
Step 7: After it has found the SDK components, it will redirect to the Welcome dialog box.



Click on Next.

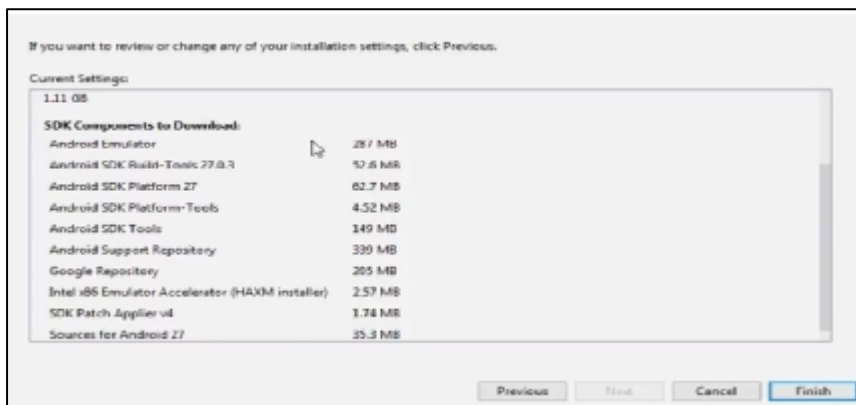


Choose Standard and click on Next. Now choose the theme, whether the Light theme or the Dark one. The light one is called the IntelliJ theme whereas the dark theme is called Dracula. Choose as required.

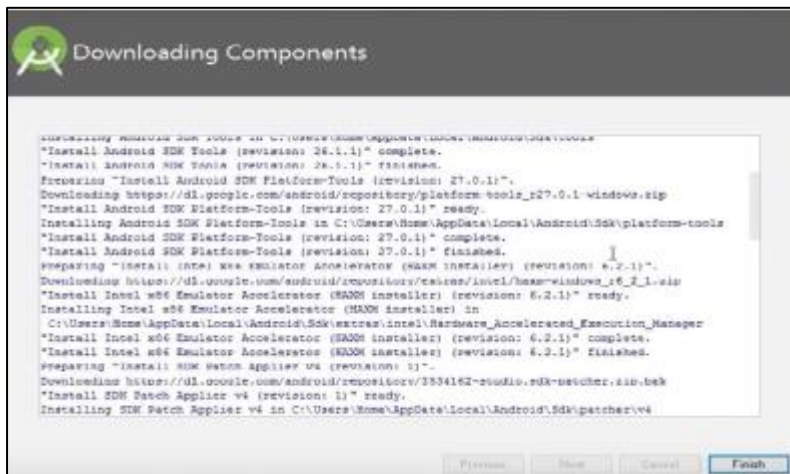


Click on the Next button.

Step 8: Now it is time to download the SDK components.

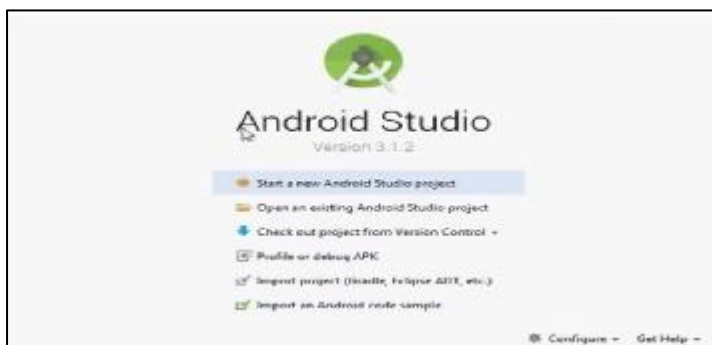


Click on Finish. Components begin to download let it complete.



The Android Studio has been successfully configured. Now it's time to launch and build apps. Click on the Finish button to launch it.

Step 9: Click on Start a new Android Studio project to build a new app.

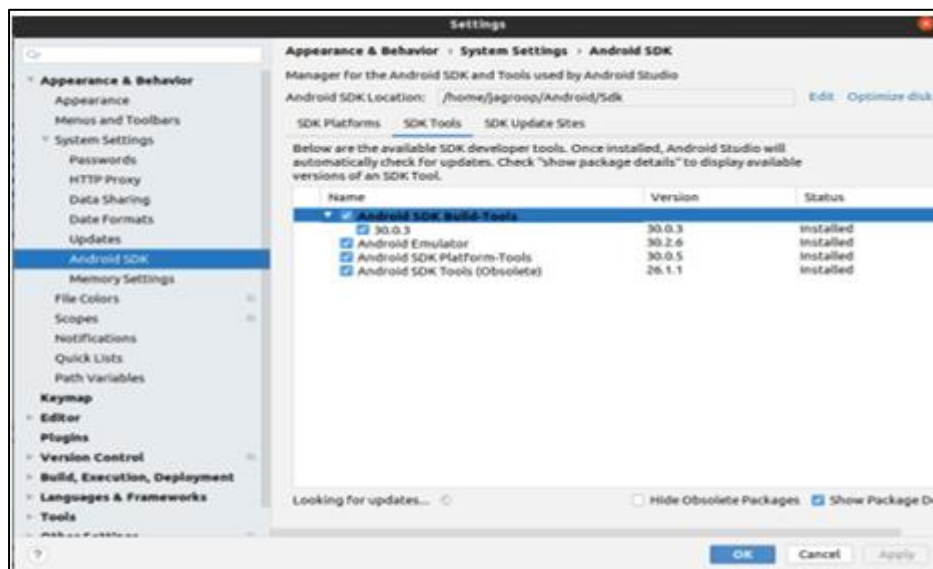


To run your first android app in Android Studio you may refer to Running your first Android app.

PRACTICAL 2.

Android SDK manager and it's all components

Android SDK is a collection of libraries and Software Development tools that are essential for Developing Android Applications. Whenever Google releases a new version or update of Android Software, a corresponding SDK also releases with it. In the updated or new version of SDK, some more features are included which are not present in the previous version. Android SDK consists of some tools which are very essential for the development of Android Application. These tools provide a smooth flow of the development process from developing and debugging. Android SDK is compatible with all operating systems such as Windows, Linux, macOS, etc.



Components of Android SDK

Android SDK Components play a major role in the Development of Android applications. Below are the important components:

1. Android SDK Tools

Android SDK tool is an important component of Android SDK. It consists of a complete set of development and debugging tools. Below are the SDK developer tools:

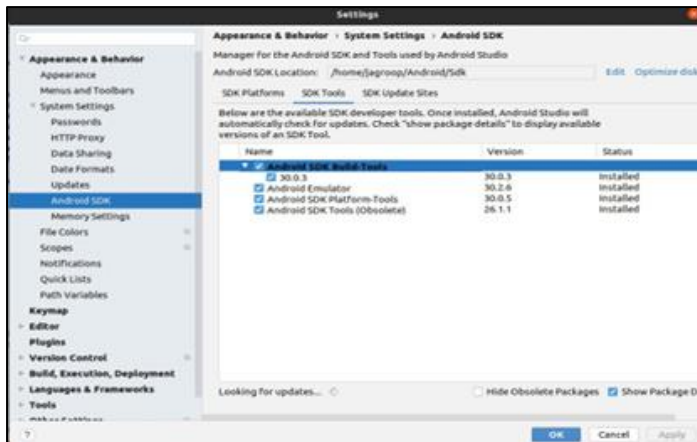
Android SDK Build tool.

Android Emulator.

Android SDK Platform-tools.

Android SDK Tools.

These are shown below:



2. Android SDK Build- Tools

Android SDK build tools are used for building actual binaries of Android App. The main functions of Android SDK Build tools are built, debug, run and test Android applications. The latest version of the Android SDK Build tool is 30.0.3. While downloading or updating Android in our System, one must ensure that its latest version is download in SDK Components.

3. Android Emulator

An Android Emulator is a device that simulates an Android device on your system. In Android Emulator the virtual android device is shown on our system on which we run the Android application that we code.



In Android Virtual Emulator all functions that are feasible on real Android mobile is works on virtual Device like:

- phone calls, text messages.
- stimulate different network speeds.
- specify the location of a device
- access on google play store and lot's more

But there is one disadvantage of this emulator is that. It is very slow when System's PC has less RAM. It works fine when a maximum GB of RAM is present on our device.

4. Android SDK Platform- tools

Android SDK Platform-tools is helpful when we are working on Project and they will show the error messages at the same time. It is specifically used for testing. It includes:

Android Debug Bridge (ADB), is a command-line tool that helps to communicate with the device. It allows us to perform an action such as Installing App and Debugging App etc.

Fastboot allows you to flash a device with a new system image.

Systrace tools help to collect and inspect timing information. It is very crucial for App Debugging.

5. Android SDK Tools

Android SDK tool is a component of SDK tool. It consists of a set of tools which and other Utilities which are crucial for the development of Android Application. It contains the complete set of Debugging and Development tools for android.

6. SDK Platforms

For Each Android Software, one SDK platform is available as shown below:

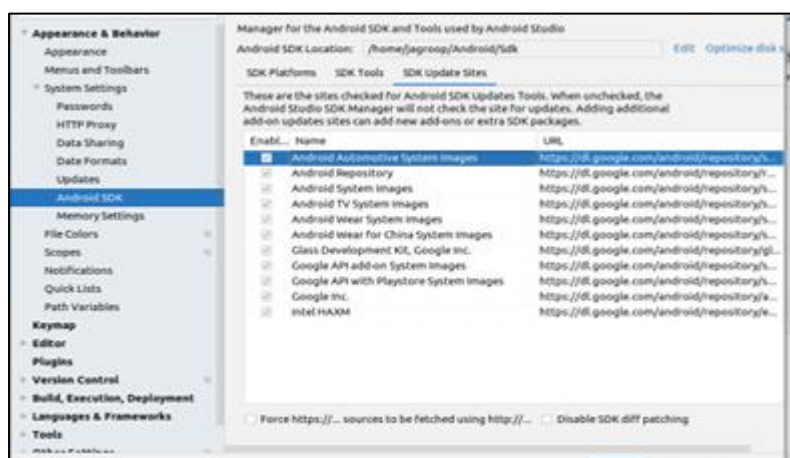


Like in this Android 11.0(R) is installed.

These are numbered according to the android version. The new version of the SDK platform has more features and more compatible but the old version is less compatible with fewer features. Like in Android 11.0(R) have more compatible and have more feature but the below versions like Android 10.0(Q), Android4.4(KitKat) have less feature and is less compatible.

7. SDK Update Sites

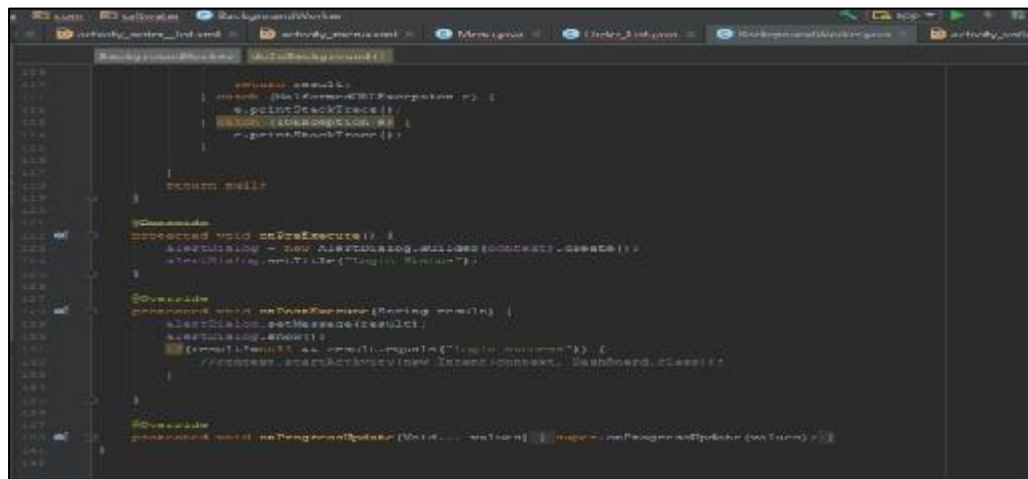
In SDK Update Sites, some sites are embedded in it which will check for Android SDK Updates Tools. In this, one must ensure we don't unclick the button below because these are checked by default which will check for updates if we will unclick it then it doesn't check updates for those.



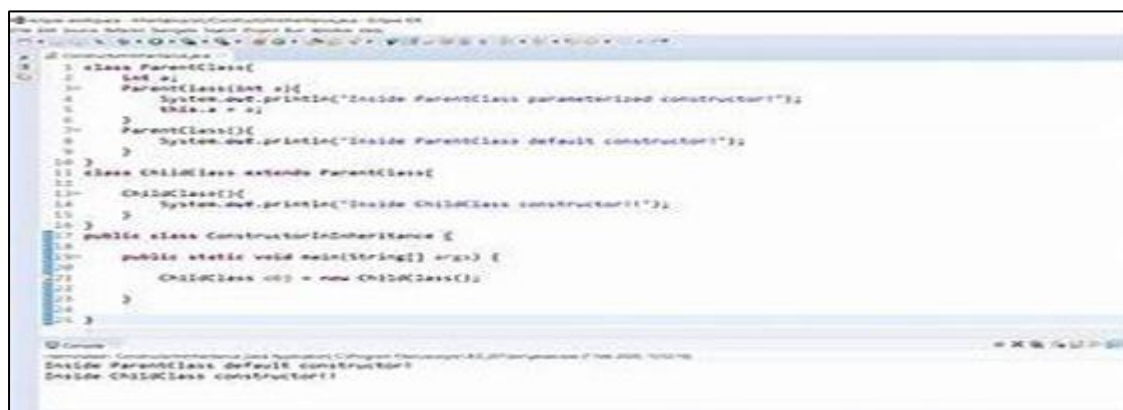
PRACTICAL NO 3.

Programs based on overriding, constructor, classes in java

Overriding: Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes. When a method in a subclass has the same name, same parameters or signature, and same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to override the method in the super-class.



Constructor: In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling the constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called.



PRACTICAL NO 4.

Programs based on the Final, this and static keyword in java.

final keyword:

Final keyword defines itself that once final keyword is used then one cannot extend or change its value. In java the final keyword is used in different methods to define any variable that can be only assigned one time in program.

Use of final keyword:

The final keyword has mainly three uses one of them is to create final class. Second one is to use final methods and third one is to use final data member. Following are uses final keyword:

1. Stop Inheritance.
2. Stop Method overriding.
3. Stop value change.

```
class Test {  
    final static int x;  
  
    static {  
        x = 10;  
    } public static void main(String[] args) {  
        System.out.println(x);  
    } }  
}
```

Static keyword:

static keyword is used java for memory management. Static variables are usually stored in static memory. Static variables are rarely used other than being declared as constants.

We can use static keyword with methods, variable, blocks and nested class. The static keyword belongs to the class than instance of the class.

static can be:

- a) Method
- b) Variable

c) Block

d) Nested class class

```
Test {
```

```
    // static method
```

```
    static void m1() {
```

```
        System.out.println("from m1");
```

```
    } public static void main(String[] args) {
```

```
        // calling m1 without creating
```

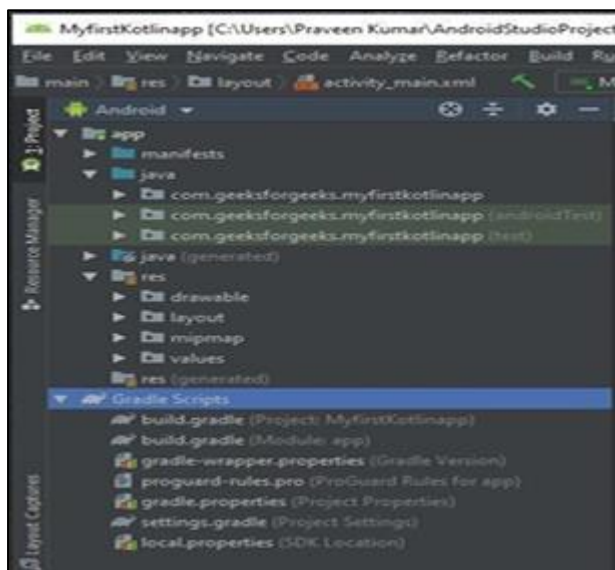
```
        // any object of class Test m1();
```

```
    } }
```

PRACTICAL NO 5.

Directory structure of an android project, common default resources folders, the values of folders, leveraging android XML

After completing the setup of Android Architecture we can create an android application in the studio. We need to create a new project for each sample application and we should understand the folder structure. It looks like this:



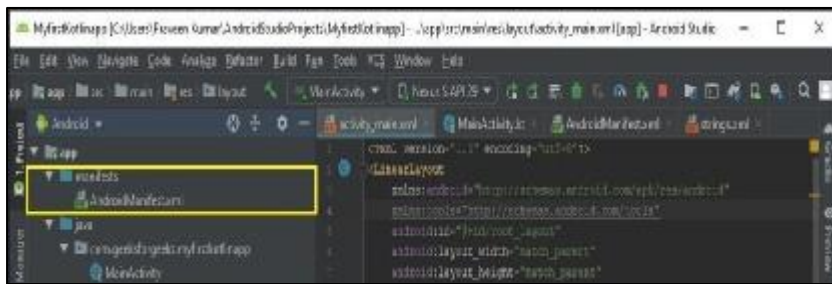
The android project contains different types of app modules, source code files, and resource files. We will explore all the folders and files in the android app.

1. Manifests Folder
2. Java Folder
3. res (Resources) Folder
 - Drawable Folder
 - Layout Folder
 - Mipmap Folder
 - Values Folder
 - Gradle Scripts

Manifests Folder:

Manifests folder contains AndroidManifest.xml for creating our android application. This file contains information about our application such as the Android version, metadata, states package for Kotlin file, and other application components. It acts as an intermediary between android OS and our

application. Following is the manifests folder structure in the android application.



AndroidManifest.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.geeksforgEEKS.myapplication">
```

```
<application
```

```
    Android:allowBackup="true"
```

```
    Android:icon="@mipmap/ic_launcher"
```

```
    Android:label="@string/app_name"
```

```
    Android:roundIcon="@mipmap/ic_launcher_round"
```

```
    Android:supportRtl="true"
```

```
    Android:theme="@style/AppTheme">
```

```
        <activity android:name=".MainActivity">
```

```
            <intent-filter>
```

```
                <action android:name="android.intent.action.MAIN"/>
```

```
                <category android:name="android.intent.category.LAUNCHER"/>
```

```
            </intent-filter>
```

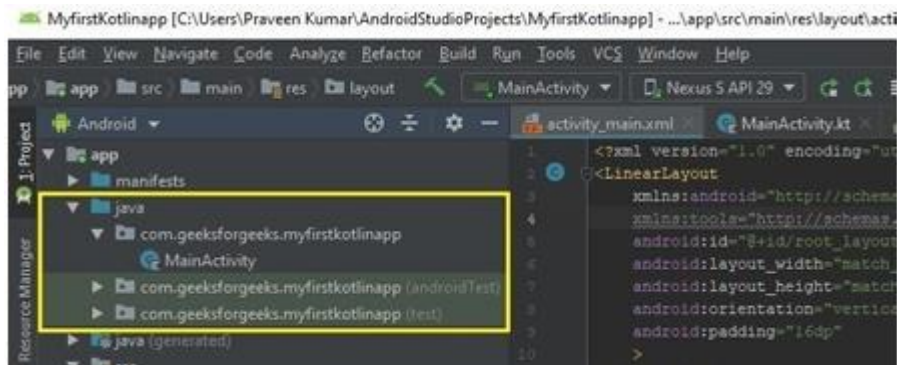
```
        </activity>
```

```
    </application>
```

```
</manifest>
```

Java folder

The Java folder contains all the java and Kotlin source code (.java) files that we create during the app development, including other Test files. If we create any new project using Kotlin, by default the class file MainActivity.kt file will create automatically under the package name “com.geeksforgeeks.myfirstkotlinapp” as shown below.



MainActivity.kt and MainActivity.java

Kotlin

```
import androidx.appcompat.app.AppCompatActivity import android.os.Bundle
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
    {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Java

```
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
```

@Override

```
protected void onCreate(Bundle savedInstanceState)
```

```
{
```

```
    super.onCreate(savedInstanceState);
```

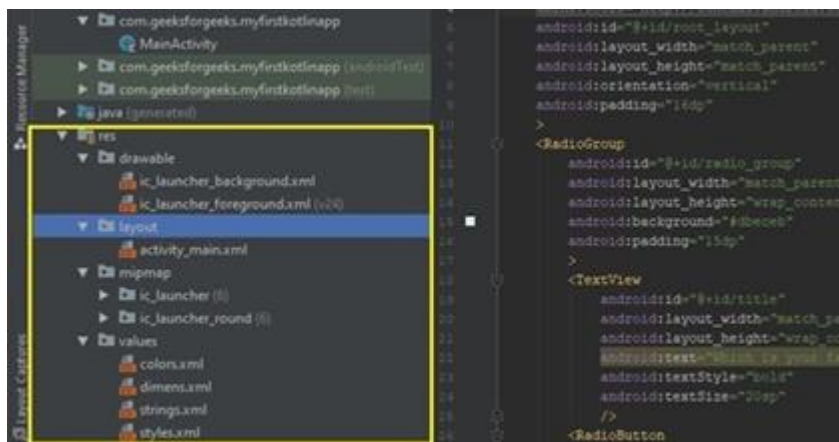
```
    setContentView(R.layout.activity_main);
```

```
}
```

```
}
```

Resource (res) folder

The resource folder is the most important folder because it contains all the non-code sources like images, XML layouts, and UI strings for our android application.



res/drawable folder

It contains the different types of images used for the development of the application. We need to add all the images in a drawable folder for the application development.

res/layout folder

The layout folder contains all XML layout files which we used to define the user interface of our application. It contains the activity_main.xml file.

XML

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<android.constraintlayout.widget.ConstraintLayout
```



```

xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MainActivity">
<TextView
    android:layout_width=wrap_content"
    android:layout_width="wrap_content"
    android:text="hello world!"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintRight_toRightOf="parent"
    app:layout_constraintTop_toTopOf="parent"/>
</android.constraintlayout.widget.ConstraintLayout>

```

res/mipmap folder

This folder contains launcher.xml files to define icons that are used to show on the home screen. It contains different density types of icons depending upon the size of the device such as hdpi, mdpi, xhdpi.

res/values folder

Values folder contains a number of XML files like strings, dimensions, colors, and style definitions. One of the most important files is the strings.xml file which contains the resources.

XML

<resources>

<string name="app_name">NameOfTheApplication</string>

<string name="checked">Checked</string>

<string name="unchecked">Unchecked</string>

</resources>

Gradle Scripts folder

Gradle means automated build system and it contains a number of files that are used to define a build configuration that can be applied to all modules in our application. In build.gradle (Project) there are build scripts and in build.gradle (Module) plugins and implementations are used to build configurations that can be applied to all our application modules.



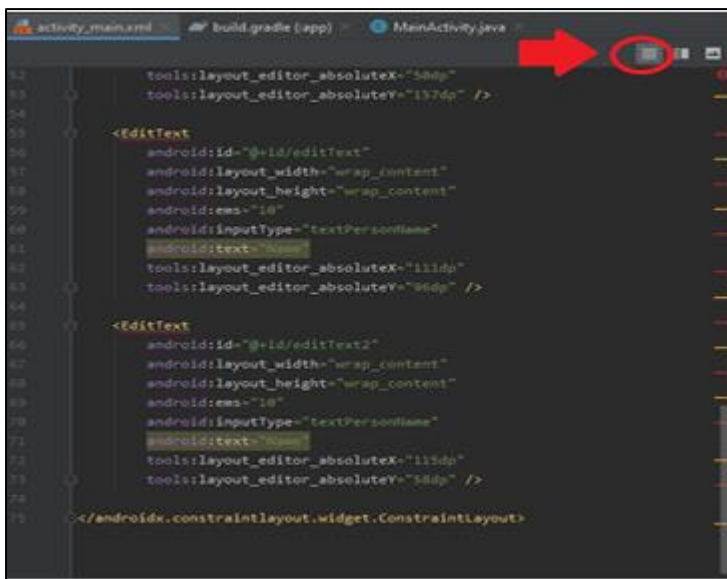
PRACTICAL 6.

Applications based on Text boxes and Button

Text boxes:

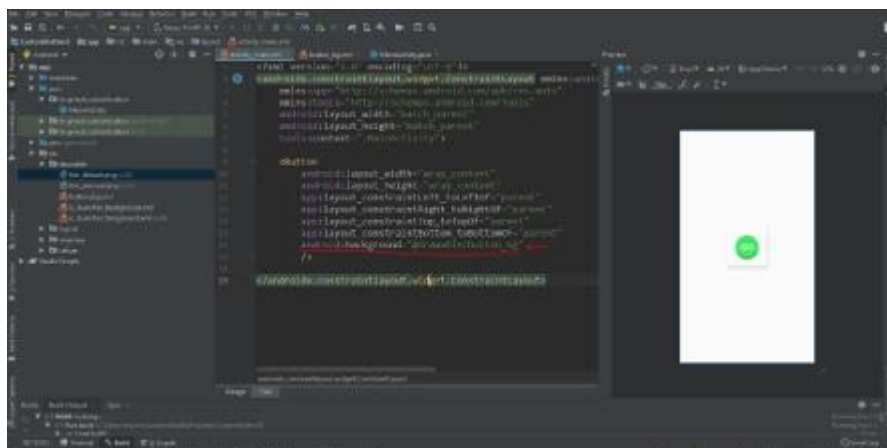
In your Android application, you may want to accept user input. There are many ways you can do that and one of the most basics is the textbox.

In Android you can use the EditText class in order to create a textbox on your screen in which the user will write text. Then, we have to bundle this textbox with a specific key event, so when the user presses the key the application can get the text written in the textbox as an input and perform a specific action.



Button:

In Android applications, a Button is a user interface that is used to perform some action when clicked or tapped. It is a very common widget in Android and developers often use it. This article demonstrates how to create a button in Android Studio.



PRACTICAL 7:-

Applications based on check boxes and buttons

A check box is used to select or deselect action items. It can be used for a single item or for a list of multiple items that a user can choose from. The control has three selection states: unselected, selected, and indeterminate. Use the indeterminate state when a collection of sub-choices have both unselected and selected states.

```
<?xml version="1.0" encoding="utf-8?>
```

```
<LinearLayout xmlns:android=http://schemas.android.com/apk/res/android
```

```
    android:orientation="vertical"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    <CheckBox android:id="@+id/checkbox_meat"
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="Meat"/>
```

```
    <CheckBox android:id="@+id/checkbox_cheese"
```

```
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="cheese"/>
```

```
</LinearLayout>
```



PRACTICAL 8:-

Applications based on radio buttons

Android radio button is a widget that can have more than one option to choose from. The user can choose only one option at a time. Each option here refers to a radio button and all the options for the topic are together referred to as Radio Group. Hence, Radio Buttons are used inside a RadioGroup.

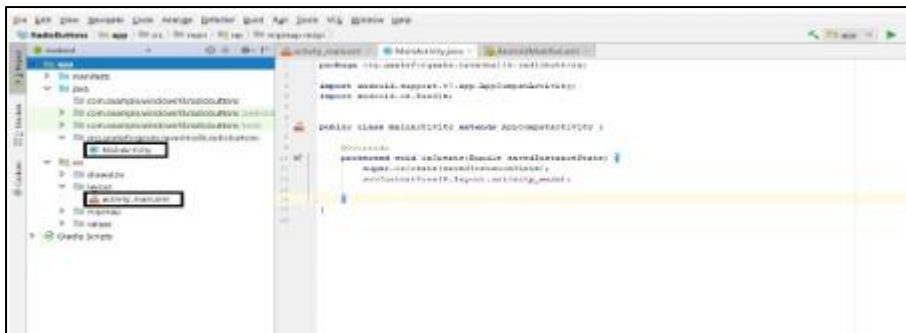
Pre-requisites:

- Android App Development Fundamentals for Beginners
- Guide to Install and Set up Android Studio
- Android | Starting with the first app/android project
- Android | Running your first Android app

Create an Android App to use Radio Buttons

This example will help in developing an Android App that creates Radio Buttons according to the above example:

Step 1: First create a new Android Application. This will create an XML file “activity_main.xml” and a Java File “MainActivity.java”. Please refer the pre requisites to learn more about this step.



Step 2: Open the “activity_main.xml” file and add the following widgets in a Relative Layout:

A TextView to display the question message

A RadioGroup to hold the option Radio Buttons which are the possible answers.

4 RadioButtons to hold an answer each.

A Submit and a Clear button to store the response.

Also, Assign the ID to each of the components along with other attributes as shown in the given image and the code below. The assigned ID on a component helps that component to be easily found and used in the Java files.

Syntax:

```
android:id="@+id/id_name"
```

Here the given IDs are as follows:

RadioGroup: groupradio

RadioButton1: radia_id1

RadioButton2: radia_id2

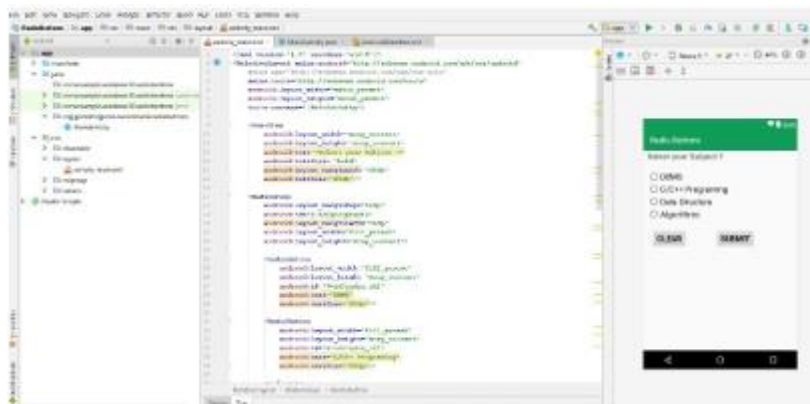
RadioButton3: radia_id3

RadioButton4: radia_id4

Submit Button: submit

Clear Button: clear

This will make the UI of the Application.



Step 3: Now, after the UI, this step will create the Backend of Application. For this, open the “MainActivity.java” file and instantiate the components made in the XML file (RadioGroup, TextView, Clear, and Submit Button) using findViewById() method. This method binds the created object to the UI Components with the help of the assigned ID.

Syntax: General

ComponentTypeobject=(ComponentType)findViewById(R.id.IdOfTheComponent);

Syntax: Components used

Button submit = (Button)findViewById(R.id.submit);

Button clear = (Button)findViewById(R.id.clear);

RadioGroup radioGroup = (RadioGroup)findViewById(R.id.groupradio);

Step 4: This step involves setting up the operations on the RadioGroup, RadioButtons and the Submit and Clear Buttons.

These operations are as follows:

Unset all the Radio Buttons initially as the default value. This is done by the following command:

```
radioGroup.clearCheck();
```

Add the Listener on the RadioGroup. This will help to know whenever the user clicks on any Radio Button, and the further operation will be performed. The listener can be added as follows:

```
radioGroup.setOnCheckedChangeListener(new
```

```
radioGroup.OnCheckedChangeListener() {}
```

Define the operations to be done when a radio button is clicked. This involves getting the specific radio button that has been clicked, using its id. Then this radio button gets set and the rest of the radio button is reset.

Add the listener on Submit button and clear button. This will be used to check when the user clicks on the button. This is done as follows:

```
submit.setOnClickListener(new View.OnClickListener() {}
```

```
clear.setOnClickListener(new View.OnClickListener() {}
```

In the Submit Button Listener, set the operations to be performed. This involves displaying the marked answer in the form of Toast.

In the Clear Button Listener, set the operations to be performed. This involves resetting all the radio buttons.

Step5: Now run the app and operate as follows:

- When the app is opened, it displays a question with 4 answers and a clear and submit button.
- When any answer is clicked, that radio button gets set.
- Clicking on any other radio button sets that one and resets the others.
- Clicking on Submit button displays the currently marked answer as a Toast.
- Clicking on Clear button resets all the radio buttons to their default state.

PRACTICAL 9:-

Applications based on Intents and Intent filters

The intent is a messaging object which tells what kind of action to be performed. The intent's most significant use is the launching of the activity. Intent facilitates the communication between the components.

Fundamental use case of Intents Starting Activity

An activity represents the single screen in an app, Bypassing intent instance we can start an activity.

Starting a Service

A Service is a component that performs operations in the background without a user interface, which is also called a background process.

Delivering a Broadcast

A broadcast is a message that any app can receive. In android, the system delivers various broadcast system events like device starts charging, disable or enable airplane mode, etc.

Intent Type

There are two types of intent

Explicit intent: Explicit intent can do the specific application action which is set by the code like changing activity, In explicit intent user knows about all the things like after clicking a button which activity will start and Explicit intents are used for communication inside the application.

Implicit Intent: Implicit intents do not name a specific component like explicit intent, instead declare general action to perform, which allows a component from another app to handle.

For example: when you tap the share button in any app you can see the Gmail, Bluetooth, and other sharing app options.

Intent Filter

- Implicit intent uses the intent filter to serve the user request.
- The intent filter specifies the types of intents that an activity, service, or broadcast receiver can respond.
- Intent filters are declared in the Android manifest file.
- Intent filter must contain <action>

XML

iiiiiiiiiiiiiiiiiiiiiiiiiiiiiiii